

MusicLang — Plano Completo de Implementação

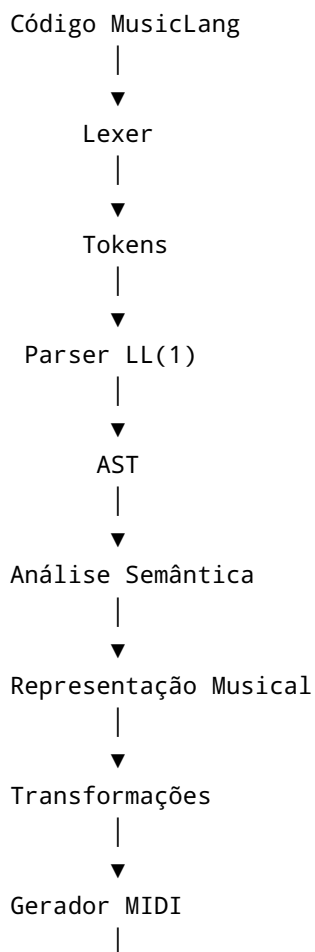
1. Objetivo

Desenvolver manualmente um compilador para uma linguagem específica de domínio (DSL) voltada à descrição, validação, transformação e geração de músicas.

A linguagem, denominada provisoriamente **MusicLang**, deverá permitir que o usuário descreva uma música por meio de uma sintaxe própria, contendo informações como nome, BPM, tonalidade, melodia, duração, pausas, acordes, instrumentos e transformações.

O compilador deverá receber um arquivo `.music`, analisar sua estrutura, validar suas regras, construir uma representação intermediária, aplicar transformações musicais e gerar um arquivo `.mid`.

Fluxo geral:



▼
arquivo .mid

O projeto deve priorizar os conceitos da disciplina de Compiladores. A geração do MIDI será a aplicação prática do compilador, e não o objetivo isolado.

2. Princípios da implementação

A implementação deverá seguir estas regras:

1. O lexer será implementado manualmente.
2. O parser será implementado manualmente.
3. A análise sintática deverá utilizar LL(1).
4. Não utilizar ANTLR, Bison, Yacc, Lark, PLY ou geradores equivalentes.
5. A BNF deverá ser projetada considerando LL(1) desde o início.
6. O parser não deverá gerar MIDI diretamente.
7. A AST deverá separar a análise sintática da análise semântica.
8. A representação musical deverá ser independente da sintaxe da linguagem.
9. As transformações musicais deverão operar sobre a representação interna.
10. A geração MIDI deverá ocorrer somente depois que a entrada estiver validada.
11. Cada etapa deverá possuir testes independentes.
12. O desenvolvimento deverá ser incremental: cada etapa deve funcionar antes do início da próxima.

3. Tecnologias

Linguagem principal:

- Python 3

Ferramentas:

- Git
- Python `venv`
- pytest
- Mido
- FluidSynth

Instalação no Linux:

```
sudo apt update
sudo apt install python3 python3-venv python3-pip git fluidsynth
```

Criar o projeto:

```
mkdir musiclang
cd musiclang

python3 -m venv .venv
source .venv/bin/activate

pip install mido pytest
```

Salvar as dependências:

```
pip freeze > requirements.txt
```

Inicializar Git:

```
git init
```

Criar `.gitignore`:

```
.venv/
__pycache__/
.pytest_cache/
*.pyc
output/
```

4. Estrutura do projeto

Criar a seguinte estrutura:

```
musiclang/
├── src/
│   ├── __init__.py
│   ├── lexer.py
│   ├── parser.py
│   ├── ast.py
│   ├── semantic.py
│   ├── music.py
│   ├── transformer.py
│   ├── midi.py
│   ├── errors.py
│   └── main.py
├── tests/
│   └── lexer/
```

```
|   ├── parser/
|   ├── semantic/
|   ├── transformer/
|   └── integration/
|
|   ├── examples/
|   │   ├── simple.music
|   │   ├── melody.music
|   │   └── variations.music
|   |
|   ├── docs/
|   │   ├── requirements.md
|   │   ├── language.md
|   │   ├── grammar.md
|   │   ├── first-follow.md
|   │   ├── architecture.md
|   │   └── errors.md
|   |
|   ├── output/
|   ├── requirements.txt
|   ├── README.md
|   └── .gitignore
```

5. Escopo da linguagem

A primeira versão da MusicLang deverá suportar:

- programa musical;
- nome da música;
- BPM;
- tonalidade;
- modo maior e menor;
- melodia;
- notas;
- oitavas;
- duração;
- pausas;
- acordes;
- tracks;
- instrumentos;
- transposição;
- mudança de oitava;
- repetição;
- reversão;
- inversão;
- geração de variações;
- saída MIDI.

Não implementar todos os recursos simultaneamente.

A ordem de desenvolvimento deverá ser incremental.

6. Sintaxe inicial

Um programa mínimo deverá ter a seguinte estrutura:

```
music "Minha Musica" {  
    tempo 120  
    key C major  
  
    melody {  
        C4 1/4  
        D4 1/4  
        E4 1/4  
        G4 1/2  
    }  
  
    output "minha_musica.mid"  
}
```

Esse programa representa:

- música chamada `Minha Musica`;
 - BPM igual a 120;
 - tonalidade C maior;
 - uma melodia;
 - quatro eventos musicais;
 - geração de `minha_musica.mid`.
-

7. Funcionalidades da linguagem

7.1 Música

```
music "Nome" {  
    ...  
}
```

7.2 BPM

```
tempo 120
```

Regra semântica inicial:

20 <= BPM <= 300

7.3 Tonalidade

key C major

ou:

key A minor

Notas inicialmente suportadas:

C
C#
D
D#
E
F
F#
G
G#
A
A#
B

Posteriormente podem ser adicionados bemóis.

7.4 Notas

C4 1/4
D4 1/4
E4 1/2

Uma nota possui:

pitch + octave + duration

7.5 Pausas

```
rest 1/4
```

7.6 Harmonia

```
harmony {  
  Cmaj 1  
  Am 1  
  Fmaj 1  
  G7 1  
}
```

7.7 Tracks

```
track piano {  
  melody {  
    C4 1/4  
    E4 1/4  
    G4 1/2  
  }  
}
```

7.8 Transformações

```
variation {  
  transpose +2  
}
```

```
variation {  
  octave +1  
}
```

```
variation {  
  reverse  
}
```

```
variation {  
  invert  
}
```

```
variation {  
    repeat 2  
}
```

8. Fase 1 — levantamento dos requisitos

Antes de programar, analisar o enunciado do projeto e registrar todos os requisitos.

Criar:

```
docs/requirements.md
```

Para cada requisito, registrar:

```
Requisito  
Descrição  
Componente responsável  
Como será demonstrado  
Teste correspondente
```

Exemplo:

```
Requisito:  
Análise sintática LL(1)  
  
Componente:  
parser.py  
  
Demonstração:  
Mostrar tokens entrando no parser e AST sendo produzida.  
  
Teste:  
tests/parser/test_music.py
```

Essa etapa deve garantir que nenhuma exigência do professor seja esquecida.

9. Fase 2 — especificação da linguagem

Criar:

docs/language.md

Definir formalmente:

- palavras reservadas;
- identificadores;
- strings;
- números;
- notas;
- oitavas;
- durações;
- operadores;
- blocos;
- comandos;
- regras semânticas;
- exemplos válidos;
- exemplos inválidos.

Antes de implementar uma funcionalidade, definir sua sintaxe.

10. Fase 3 — especificação dos tokens

Definir os tokens da linguagem.

Palavras reservadas:

```
MUSIC
TEMPO
KEY
MAJOR
MINOR
MELODY
HARMONY
TRACK
VARIATION
GENERATE
TRANSPOSE
OCTAVE
REVERSE
INVERT
REPEAT
OUTPUT
REST
```

Símbolos:

```
LBRACE  
RBRACE  
LBRACKET  
RBRACKET  
LPAREN  
RPAREN  
PLUS  
MINUS  
COMMA
```

Dados:

```
NUMBER  
STRING  
NOTE  
IDENTIFIER  
DURATION
```

Controle:

```
EOF
```

A especificação deverá indicar exatamente como cada token é reconhecido.

11. Fase 4 — implementação do Lexer

Implementar manualmente um analisador léxico em:

```
src/lexer.py
```

O lexer deverá percorrer o código fonte caractere por caractere.

Cada token deverá conter:

```
tipo  
lexema  
linha  
coluna
```

Exemplo:

tempo 120

deverá produzir conceitualmente:

TOKEN_TEMPO
TOKEN_NUMBER(120)

Para:

C4 1/4

deverá produzir tokens correspondentes à nota e à duração.

O lexer deverá ignorar:

- espaços;
- tabs;
- quebras de linha quando não forem semanticamente relevantes;
- comentários.

O lexer deverá detectar caracteres inválidos.

12. Fase 5 — testes do Lexer

Criar testes para:

tempo 120

key C major

music "Teste"

C4 1/4

rest 1/2

Também criar testes inválidos.

Exemplo:

tempo @

Resultado esperado:

LexerError

Nenhuma etapa posterior deverá depender de um lexer que ainda possua erros básicos.

13. Fase 6 — construção da BNF

Somente depois de definir os tokens e a sintaxe geral construir a BNF.

Uma estrutura inicial será:

```
<program> ::= <music>

<music> ::= MUSIC STRING LBRACE
           <music_body>
           RBRACE
```

Depois deverão ser criadas as produções para:

```
tempo
key
melody
notes
rests
harmony
tracks
variations
output
```

A gramática deverá ser construída pensando desde o início em LL(1).

Evitar:

- recursão à esquerda;
 - ambiguidades;
 - produções com conflitos de FIRST;
 - decisões que exijam mais de um token de lookahead.
-

14. Fase 7 — FIRST e FOLLOW

Para cada não-terminal calcular:

FIRST(X)

e:

FOLLOW(X)

Documentar os cálculos em:

docs/first-follow.md

Não utilizar inicialmente um programa para fazer os cálculos automaticamente.

Primeiro realizar os cálculos manualmente para compreender a gramática.

Depois, se desejado, criar uma ferramenta auxiliar para verificar os resultados.

15. Fase 8 — tabela LL(1)

Com FIRST e FOLLOW, construir a tabela:

M[A, a]

onde:

- A é um não-terminal;
- a é o próximo token.

A tabela deverá possuir no máximo uma produção em cada célula.

Se uma célula possuir mais de uma produção, a gramática possui conflito e deverá ser modificada.

Documentar a tabela em:

docs/grammar.md

16. Fase 9 — implementação do Parser LL(1)

Implementar manualmente em:

```
src/parser.py
```

O parser deverá utilizar:

```
pilha  
token atual  
tabela LL(1)
```

Fluxo:

```
Tokens  
↓  
Pilha do parser  
↓  
Tabela LL(1)  
↓  
Produção  
↓  
AST
```

O parser não deverá gerar MIDI.

Sua responsabilidade será somente analisar a estrutura sintática e construir a AST.

17. Fase 10 — tratamento de erros sintáticos

Criar:

```
ParserError
```

O erro deverá informar:

- linha;
- coluna;
- token encontrado;
- token esperado;
- contexto quando possível.

Exemplo:

Erro sintático na linha 7, coluna 12.

Esperado:

DURATION

Encontrado:

IDENTIFIER("abc")

Implementar recuperação de erro quando isso não complicar excessivamente o projeto.

Inicialmente, um erro pode interromper a compilação.

18. Fase 11 — construção da AST

Criar:

```
src/ast.py
```

A AST deverá representar conceitos da linguagem e não tokens.

Estrutura conceitual:

```
Music
├── name
├── tempo
├── key
├── melody
│   ├── Note
│   ├── Note
│   ├── Rest
│   └── Note
├── harmony
└── variations
```

Possíveis classes:

```
Music
Melody
Note
Rest
Chord
Track
Variation
Transpose
```

```
OctaveShift
Reverse
Invert
Repeat
Output
```

A AST deverá ser fácil de percorrer e modificar.

19. Fase 12 — visualização da AST

Implementar uma forma de imprimir a AST.

Exemplo:

```
Music(name="Minha Musica")
├─ Tempo(120)
├─ Key(C, major)
├─ Melody
│   ├── Note(C4, 1/4)
│   ├── Note(D4, 1/4)
│   ├── Note(E4, 1/4)
│   └─ Note(G4, 1/2)
└─ Output("minha_musica.mid")
```

Criar uma opção de CLI:

```
python -m src.main --ast examples/simple.music
```

Isso será útil tanto para depuração quanto para apresentação.

20. Fase 13 — análise semântica

Criar:

```
src/semantic.py
```

A análise semântica deverá validar regras que não podem ser expressas adequadamente pela BNF.

Exemplos:


```
tempo 500
```

é sintaticamente correto, mas semanticamente inválido.

Verificar:

- BPM;
- notas;
- oitavas;
- durações;
- tonalidades;
- acordes;
- comandos incompatíveis;
- valores fora dos limites;
- duplicações quando proibidas;
- transformações válidas.

Criar:

```
SemanticError
```

21. Fase 14 — modelo musical interno

Criar:

```
src/music.py
```

Esse módulo deverá representar a música de forma independente da linguagem.

Uma nota poderá possuir:

```
pitch
duration
velocity
start_time
```

O pitch deverá poder ser representado como número MIDI.

Exemplos:

```
C4 → 60
C#4 → 61
D4 → 62
```

```
E4 → 64  
A4 → 69
```

A representação interna deverá permitir transformação sem depender da sintaxe textual.

22. Fase 15 — representação das durações

Adotar inicialmente uma resolução de:

```
PPQ = 480
```

Com isso:

```
whole    = 1920 ticks  
half     = 960 ticks  
quarter  = 480 ticks  
eighth   = 240 ticks  
sixteenth = 120 ticks
```

A representação interna deverá converter as durações da linguagem para ticks.

Isso facilitará a geração do MIDI.

23. Fase 16 — primeiro MIDI

Criar:

```
src/midi.py
```

O gerador deverá receber a representação musical interna e produzir:

```
.mid
```

Primeiro implementar somente:

- uma track;
- tempo;
- notas;
- duração;
- velocity;
- instrumento opcional.

O primeiro objetivo é fazer funcionar:

```
music "Hello" {  
  tempo 120  
  key C major  
  
  melody {  
    C4 1/4  
    D4 1/4  
    E4 1/4  
    G4 1/2  
  }  
  
  output "hello.mid"  
}
```

Resultado:

```
hello.mid
```

Esse é o primeiro marco de integração completa.

24. Fase 17 — teste auditivo

Utilizar FluidSynth ou outro reprodutor MIDI para verificar os arquivos gerados.

O objetivo é validar:

- pitch;
- duração;
- tempo;
- ordem das notas;
- pausas.

O MIDI não deverá ser considerado correto apenas porque o arquivo foi criado. Ele precisa produzir o resultado musical esperado.

25. Fase 18 — transformação de transposição

Implementar:

```
variation {  
    transpose +2  
}
```

A transformação deverá ocorrer sobre o pitch MIDI.

Exemplo:

```
C4 = 60
```

com:

```
transpose +2
```

resultará em:

```
D4 = 62
```

A transformação deverá ser independente do parser.

Arquitetura:

```
AST  
↓  
Modelo Musical  
↓  
Transpose  
↓  
Modelo Musical modificado  
↓  
MIDI
```

26. Fase 19 — mudança de oitava

Implementar:

```
octave +1
```

A transformação equivale a adicionar 12 semitons:

```
C4 → C5  
60 → 72
```

Validar limites de pitch MIDI.

27. Fase 20 — repetição

Implementar:

```
repeat 2
```

A música ou trecho deverá ser duplicado mantendo a sequência temporal correta.

Exemplo:

```
C4  
D4  
E4
```

com:

```
repeat 2
```

resulta em:

```
C4  
D4  
E4  
C4  
D4  
E4
```

28. Fase 21 — reverse

Implementar:

```
reverse
```

A sequência melódica deverá ser invertida.

Exemplo:

```
C4 D4 E4 G4
```

torna-se:

```
G4 E4 D4 C4
```

A duração e o posicionamento temporal deverão ser tratados corretamente.

29. Fase 22 — inversão melódica

Implementar:

```
invert
```

A inversão deverá utilizar um eixo definido.

Exemplo conceitual:

```
melodia:  
C D E G
```

poderá ser invertida em torno de C:

```
C Bb Ab F
```

A regra matemática deverá ser documentada antes da implementação.

30. Fase 23 — geração de variações

Implementar:

```
generate {  
    transpose [-2, 2, 5]  
}
```

O compilador deverá criar uma cópia da representação musical para cada variação.

Exemplo:

```
output/  
├─ original.mid  
├─ variation_01.mid  
├─ variation_02.mid  
└─ variation_03.mid
```

O nome dos arquivos deverá ser determinístico.

31. Fase 24 — múltiplas tracks

Adicionar suporte a:

```
track piano {  
    ...  
}  
  
track bass {  
    ...  
}
```

Cada track deverá possuir:

- instrumento;
- sequência de eventos;
- canal MIDI apropriado;
- posição temporal própria.

O gerador MIDI deverá criar múltiplas tracks.

32. Fase 25 — instrumentos

Associar instrumentos da linguagem aos programas MIDI correspondentes.

Exemplo:

```
track piano {  
    ...  
}
```

O compilador deverá converter `piano` para o programa MIDI correspondente.

Inicialmente implementar poucos instrumentos:

```
piano  
violin  
bass  
guitar  
flute  
drums
```

Posteriormente a lista poderá ser expandida.

33. Fase 26 — interface de linha de comando

Criar:

```
src/main.py
```

O compilador deverá aceitar:

```
python -m src.main examples/simple.music
```

Opções planejadas:

```
python -m src.main arquivo.music
```

Compilar e gerar MIDI.

```
python -m src.main --tokens arquivo.music
```

Mostrar tokens.

```
python -m src.main --ast arquivo.music
```

Mostrar AST.

```
python -m src.main --check arquivo.music
```

Validar sem gerar MIDI.


```
python -m src.main --output output/ arquivo.music
```

Definir diretório de saída.

34. Fase 27 — integração completa

Criar um exemplo completo:

```
music "Minha Musica" {  
    tempo 120  
    key C major  
  
    track piano {  
        melody {  
            C4 1/4  
            D4 1/4  
            E4 1/4  
            G4 1/2  
            rest 1/4  
            G4 1/4  
            E4 1/4  
            D4 1/4  
            C4 1/2  
        }  
    }  
  
    variation {  
        transpose +2  
    }  
  
    output "minha_musica.mid"  
}
```

O sistema deverá executar:

```
arquivo .music  
    ↓  
lexer  
    ↓  
tokens  
    ↓  
parser LL(1)  
    ↓  
AST  
    ↓  
análise semântica
```

```
↓  
modelo musical  
↓  
transformação  
↓  
MIDI
```

35. Fase 28 — testes

Cada componente deverá possuir testes próprios.

Estrutura:

```
tests/  
├── lexer/  
│   ├── test_keywords.py  
│   ├── test_numbers.py  
│   ├── test_notes.py  
│   └── test_errors.py  
├── parser/  
│   ├── test_music.py  
│   ├── test_melody.py  
│   └── test_variation.py  
├── semantic/  
│   ├── test_tempo.py  
│   ├── test_pitch.py  
│   └── test_duration.py  
├── transformer/  
│   ├── test_transpose.py  
│   ├── test_reverse.py  
│   └── test_repeat.py  
└── integration/  
    ├── test_simple_music.py  
    └── test_midi_generation.py
```

Executar:

```
pytest
```

36. Casos de teste obrigatórios

Caso válido mínimo

```
music "Teste" {  
  tempo 120  
  key C major  
  
  melody {  
    C4 1/4  
  }  
  
  output "teste.mid"  
}
```

BPM inválido

```
tempo 500
```

Esperar erro semântico.

Nota inválida

```
X4 1/4
```

Esperar erro léxico ou semântico, conforme a estratégia adotada.

Oitava inválida

```
C99 1/4
```

Esperar erro semântico.

Duração inválida

```
C4 abc
```

Esperar erro sintático ou léxico.

Estrutura inválida

```
music "Teste" {  
  tempo 120  
  melody {  
    C4 1/4
```

Esperar erro sintático.

Transposição

Entrada:

```
C4 1/4
```

Transformação:

```
transpose +2
```

Resultado:

```
D4
```

37. Fase 29 — documentação

Documentar:

```
README.md
```

O README deverá conter:

- objetivo;
- instalação;
- execução;
- sintaxe;
- exemplos;
- arquitetura;
- testes;
- geração MIDI.

Documentar a gramática:

```
docs/grammar.md
```

Documentar FIRST/FOLLOW:

```
docs/first-follow.md
```

Documentar arquitetura:

```
docs/architecture.md
```

Documentar erros:

```
docs/errors.md
```

38. Fase 30 — Git

Fazer commits pequenos e semanticamente claros.

Exemplos:

```
git add .  
git commit -m "chore: initialize project"
```

```
git add .  
git commit -m "feat: implement lexical tokens"
```

```
git add .  
git commit -m "feat: implement lexer"
```

```
git add .  
git commit -m "feat: add LL1 grammar"
```

```
git add .  
git commit -m "feat: implement parser"
```

```
git add .  
git commit -m "feat: add AST"
```

```
git add .  
git commit -m "feat: add semantic analysis"
```

```
git add .  
git commit -m "feat: generate MIDI"
```

39. Marcos do projeto

O projeto deverá ser considerado concluído em etapas.

Marco 1

Lexer funcionando.

Entrada:

```
tempo 120
```

Saída:

```
Tokens
```

Marco 2

Parser LL(1) funcionando.

Entrada:

```
music "Teste" {...}
```

Saída:

```
AST
```

Marco 3

Análise semântica funcionando.

Entradas inválidas devem ser rejeitadas.

Marco 4

Primeiro MIDI funcionando.

Entrada:

```
C4 D4 E4
```

Saída:

```
arquivo.mid
```

Marco 5

Transformações funcionando.

```
transpose  
octave  
reverse  
repeat  
invert
```

Marco 6

Geração de múltiplas variações.

Marco 7

Múltiplas tracks e instrumentos.

Marco 8

CLI, testes e documentação completos.

40. Ordem final de implementação

A implementação deverá seguir esta ordem:

1. Ler e decompor os requisitos do projeto
2. Definir o escopo da MusicLang
3. Definir exemplos da linguagem
4. Definir tokens

5. Implementar Lexer
6. Testar Lexer
7. Definir BNF
8. Remover ambiguidades
9. Eliminar recursão à esquerda
10. Fatorar a gramática quando necessário
11. Calcular FIRST
12. Calcular FOLLOW
13. Construir tabela LL(1)
14. Verificar ausência de conflitos
15. Implementar parser LL(1)
16. Implementar AST
17. Testar parser
18. Implementar análise semântica
19. Implementar modelo musical interno
20. Converter notas para MIDI pitch
21. Converter duração para ticks
22. Implementar primeiro gerador MIDI
23. Gerar primeira música
24. Implementar transposição
25. Implementar mudança de oitava
26. Implementar repetição
27. Implementar reverse
28. Implementar inversão
29. Implementar geração de variações
30. Implementar tracks
31. Implementar instrumentos
32. Implementar CLI
33. Criar testes de integração
34. Criar exemplos completos
35. Documentar
36. Preparar demonstração

41. Regra de desenvolvimento

Não implementar uma etapa enquanto a anterior não estiver suficientemente estável.

A regra será:

```
Especificar
↓
Implementar
↓
Testar
↓
Corrigir
↓
```


Documentar

↓

Avançar

Não tentar implementar simultaneamente:

BNF + parser + AST + MIDI

Isso dificulta encontrar erros.

O projeto deve ser construído como um compilador tradicional, mantendo as fases separadas.

42. Resultado esperado

Ao final, deverá ser possível executar:

```
python -m src.main examples/variations.music
```

e obter algo semelhante a:

MusicLang Compiler

Source: examples/variations.music

```
[1/5] Lexical analysis..... OK
[2/5] Syntax analysis..... OK
[3/5] Semantic analysis..... OK
[4/5] Transformations..... OK
[5/5] MIDI generation..... OK
```

Music:

Minha Musica

Tempo:

120 BPM

Key:

C major

Tracks:

2

Variations:

4

Output:

```
output/original.mid  
output/transpose_2.mid  
output/transpose_5.mid  
output/reverse.mid
```

Os arquivos MIDI deverão poder ser reproduzidos para demonstrar que o compilador não apenas reconheceu a linguagem, mas efetivamente **transformou o programa MusicLang em um resultado musical executável**.

43. Critério de sucesso

O projeto será considerado tecnicamente completo quando:

- a linguagem estiver formalmente especificada;
- a BNF estiver documentada;
- FIRST e FOLLOW estiverem calculados;
- a gramática for LL(1);
- a tabela LL(1) estiver documentada;
- o lexer funcionar manualmente;
- o parser funcionar manualmente;
- a AST for construída;
- a análise semântica detectar erros;
- as transformações forem executadas sobre a representação musical;
- o MIDI for gerado corretamente;
- pelo menos uma música completa puder ser compilada;
- pelo menos três variações puderem ser geradas;
- existirem testes automatizados;
- o projeto possuir documentação;
- a execução puder ser demonstrada pela linha de comando.

O foco principal da apresentação deverá ser mostrar que o arquivo MusicLang percorre todas as fases de um compilador:

```
Fonte  
↓  
Análise Léxica  
↓  
Tokens  
↓  
Análise Sintática LL(1)  
↓  
AST  
↓  
Análise Semântica  
↓  
Representação Musical  
↓
```

Transformações

↓

Geração de Código

↓

MIDI

Dessa forma, a geração musical funciona como uma aplicação concreta de um compilador completo, permitindo demonstrar tanto os conceitos teóricos da disciplina quanto um resultado final visualmente e auditivamente verificável.